Design Optimization Project 3

Reading copy · Python code hidden · Saved explanations, figures, and results preserved

Design Optimization Project 3: PINN/PINO

Project purpose and approach

This project investigates neural methods for one-dimensional Bose–Einstein condensate control, inspired by Adriazola and Goodman's squeezing problem.

We pursue Path B: Diagnosis. We compare a conventional Gross–Pitaevskii solver with a physics-informed space–time network and a matching data-only network. Time horizon is our difficulty knob. A supervised FNO provides a supplementary final-time surrogate experiment.

Our experiments investigate when predictions deteriorate and whether removing physics losses resolves the failure. We evaluate numerical consistency, prediction errors, loss components, and computational cost.

This Code is just to load tools we need

- NumPy: Handles mathematical calculations and arrays of numbers.
- SciPy: Helps solve optimization problems.
- Matplotlib: Creates graphs to visualize our results.
- PyTorch: Will be used later to build our AI model.
- Time: Helps us measure how long calculations take. Why we're using it: These libraries provide the mathematical, visualization, and machine-learning tools needed to complete the project.

```
NumPy: 2.1.3
SciPy: 1.16.3
PyTorch: 2.11.0+cpu
Environment is ready!
```

This code simulates squeezing a Bose–Einstein condensate (BEC) wave packet using the Gross–Pitaevskii equation (GPE), which is the equation discussed in J. Adriazola and R. H. Goodman research paper.

First, the code creates a simulation area and generates a Gaussian wave packet as the starting point. It then applies a control that gradually increases from 1 to 100 over 2.5 units of time. This control strengthens a potential that confines the wave, causing it to become narrower.

The code uses the split-step Fourier method to calculate how the wave changes over 2,500 time steps. It accounts for the wave's movement, the external potential, and nonlinear interactions within the condensate.

Finally, it creates a graph comparing the initial and final wave shapes and checks that the total probability remains approximately 100%.

Why we're using it for our project: This is our first traditional numerical simulation of the researcher's wave-squeezing problem. It helps us understand how applying a control changes the wave before we attempt to optimize that control and compare the results against an AI neural operator.

In our simulation, the wave's width decreased by 52.56%, showing that the applied control successfully squeezed it. However, this is a starting simulation using a Gaussian initial wave, not yet an optimized solution that reproduces the paper's full experiment

Mathematical formulation

We solve the dimensionless one-dimensional Gross–Pitaevskii equation:

$$i\psi_t = -\frac{1}{2}\psi_{xx} + \frac{1}{2}u(t)x^2\psi + |\psi|^2\psi.$$

The computational domain is $x \in [-8, 8)$ with periodic boundary conditions and 512 spatial points. The simulation uses $\Delta t = 0.001$ and final time $T = 2.5$.

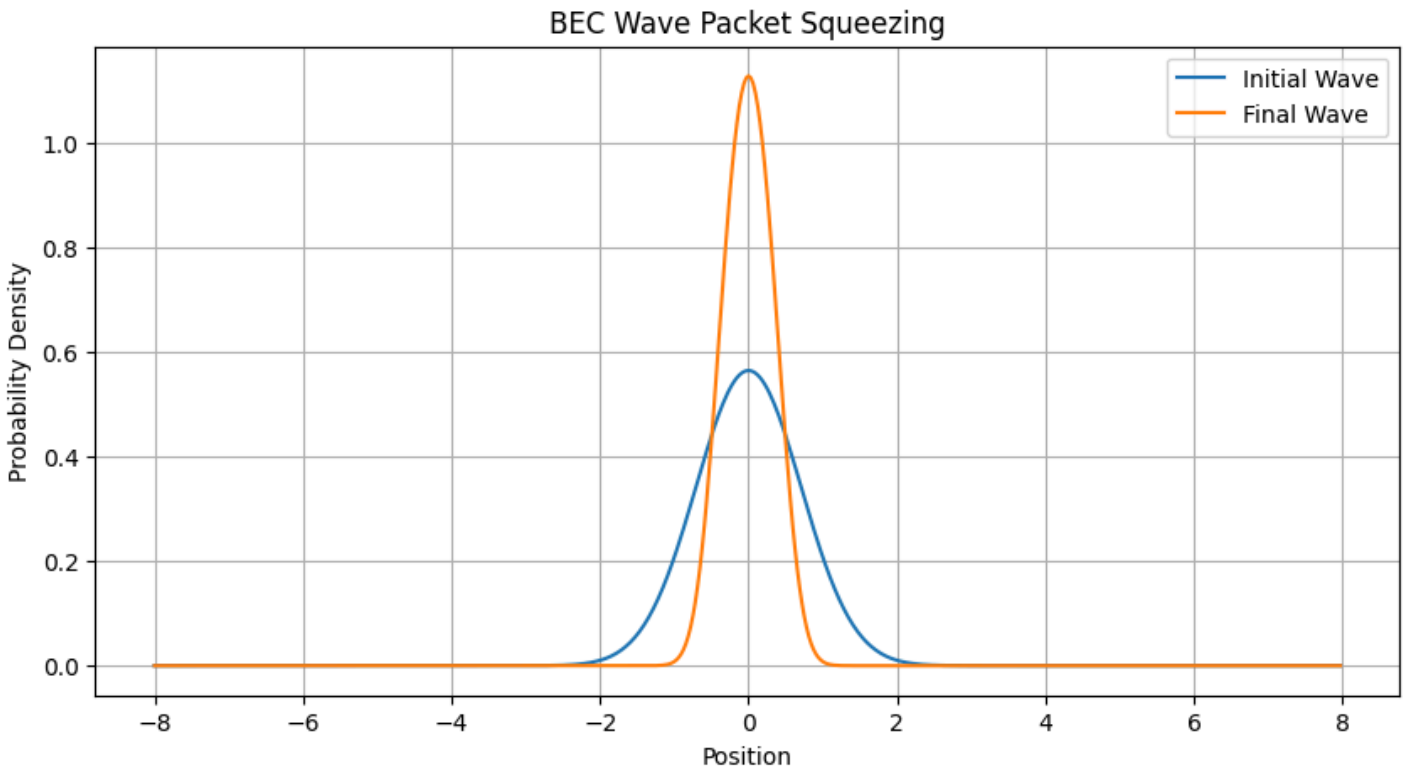
The initial state is $\psi(x, 0) = C\exp(-x^2/2)$, normalized numerically to unit probability. This Gaussian simplifies the paper's initial condition, which uses a superposition of two even modes selected through reduced-Hamiltonian minimization.

The baseline control is $u(t) = 1 + 99t/T$. Optimized controls add three sine terms:

$$u(t) = 1 + 99t/T + \sum_{n=1}^{3} a_n \sin(n\pi t/T).$$

For the matched optimization comparison, coefficients satisfy $a_n \in [-20, 20]$. These coefficient bounds do not impose the same bounds on the control itself.

Our difficulty knob is the observation/training horizon: 0.1, 0.5, 1.0, and 2.5. The control schedule remains unchanged; each experiment uses a different-length prefix of the trajectory.




```
Final probability: 1.0000000000003375
```

```
Initial Width: 0.7071
Final Width: 0.3355
Squeezing Percentage: 52.56 %
```

This code saves the results from our traditional BEC wave-squeezing simulation into a CSV file called baseline_results.csv.

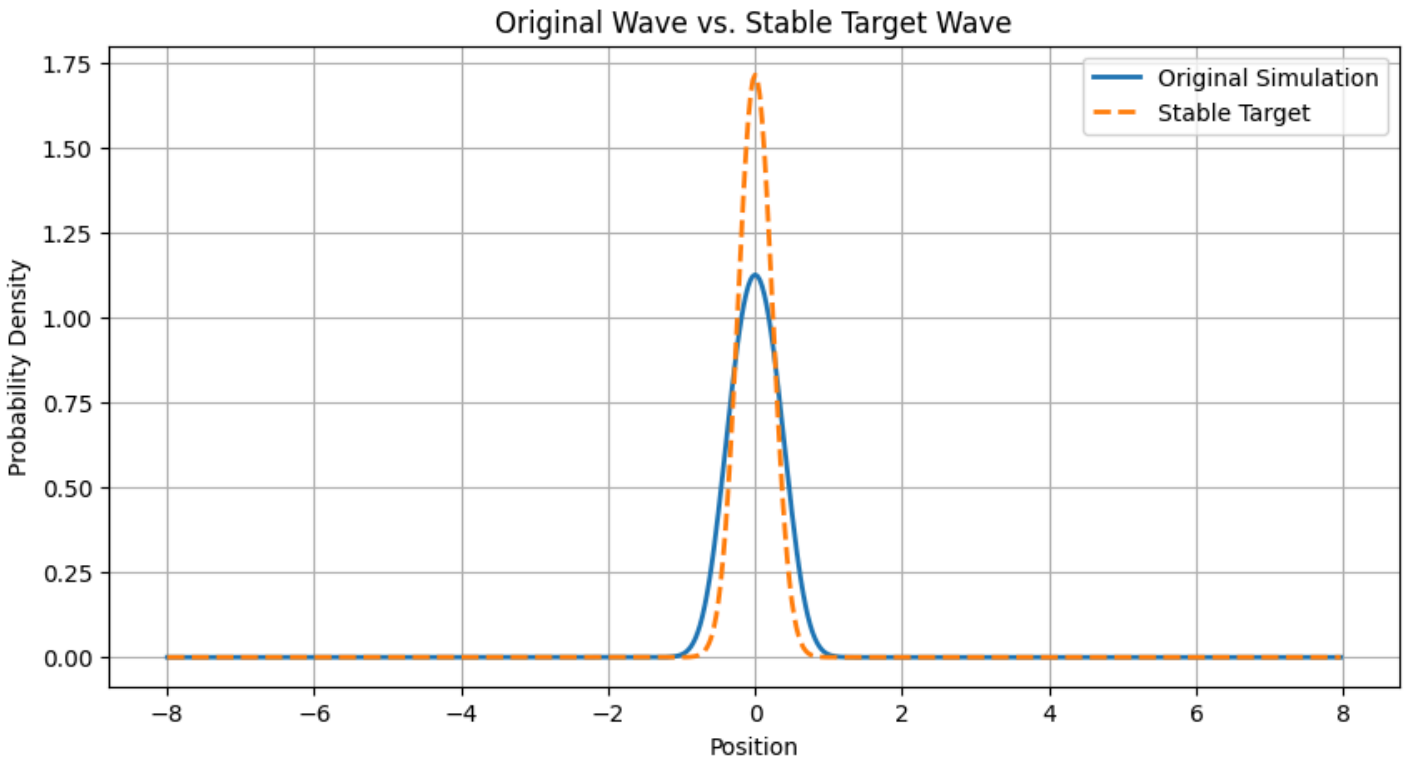
```
            Measurement      Value
0         Initial Width   0.707107
1           Final Width   0.335464
2  Squeezing Percentage  52.558214
3     Final Probability   1.000000
Results saved successfully!
```

Origional Wave VS Stable Target Wave

This code calculates a stable target wave for our Bose–Einstein condensate (BEC) using a mathematical technique called imaginary-time propagation.

First, it creates a Gaussian wave and applies the final control strength of 100. It then uses the split-step Fourier method with imaginary time to gradually move the wave toward a low-energy, stable state.

Once the calculation is finished, the code creates a graph comparing our original simulation against the stable target wave.




```
Stable target probability: 1.0
Target relative stationary residual: 0.00024064387906642669
```

The results of this code:

-The blue line represents our original squeezed wave.

-The orange dashed line represents the stable target wave.

-The orange line is taller and narrower, meaning our original simulation has not reached the desired final shape.

-The target's total probability is 1.0, confirming that it is properly normalized. Why we're using it for our project: Our researcher wants to squeeze a BEC wave while minimizing unwanted oscillations. Creating a stable target gives our optimization algorithm a more meaningful goal than simply making the wave narrower.

We can now work toward finding control settings that bring the wave closer to its desired final state

The target wave is normalized to total probability 1. Its relative stationary GPE residual is approximately (2.41\times10^{-4}), indicating that it approximately satisfies the stationary equation on our numerical grid. We will check sensitivity to the imaginary-time step before treating it as a reference target.

This next code measures how different our original squeezed wave is from the stable target wave.

First, it calculates the target wave's width. Then, it compares the probability densities of the two waves and calculates a numerical error. A smaller error means the simulated wave is closer to the target.

Finally, it prints the original wave width, target wave width, and density difference.

This gives us a measurable way to evaluate how well our original control performs. **Later**, we can compare these measurements with the results from our optimized control and eventually our AI neural operator.

The goal is to determine whether optimization can bring the final wave closer to its desired state.

```
Original final width: 0.3355
Stable target width: 0.2305
Density difference: 0.356551
```

our original wave is approximately 45.6% wider than the stable target. That means our first control successfully squeezed the wave, but it didn't bring the wave close enough to the desired final shape.

The density difference of 0.356551 gives us a starting error measurement. Our next goal is to make this number smaller

This code takes the control coefficients found by SciPy and reruns the Bose–Einstein condensate (BEC) simulation.

It compares the final optimized state with the desired target using probability density error and wave-function infidelity.

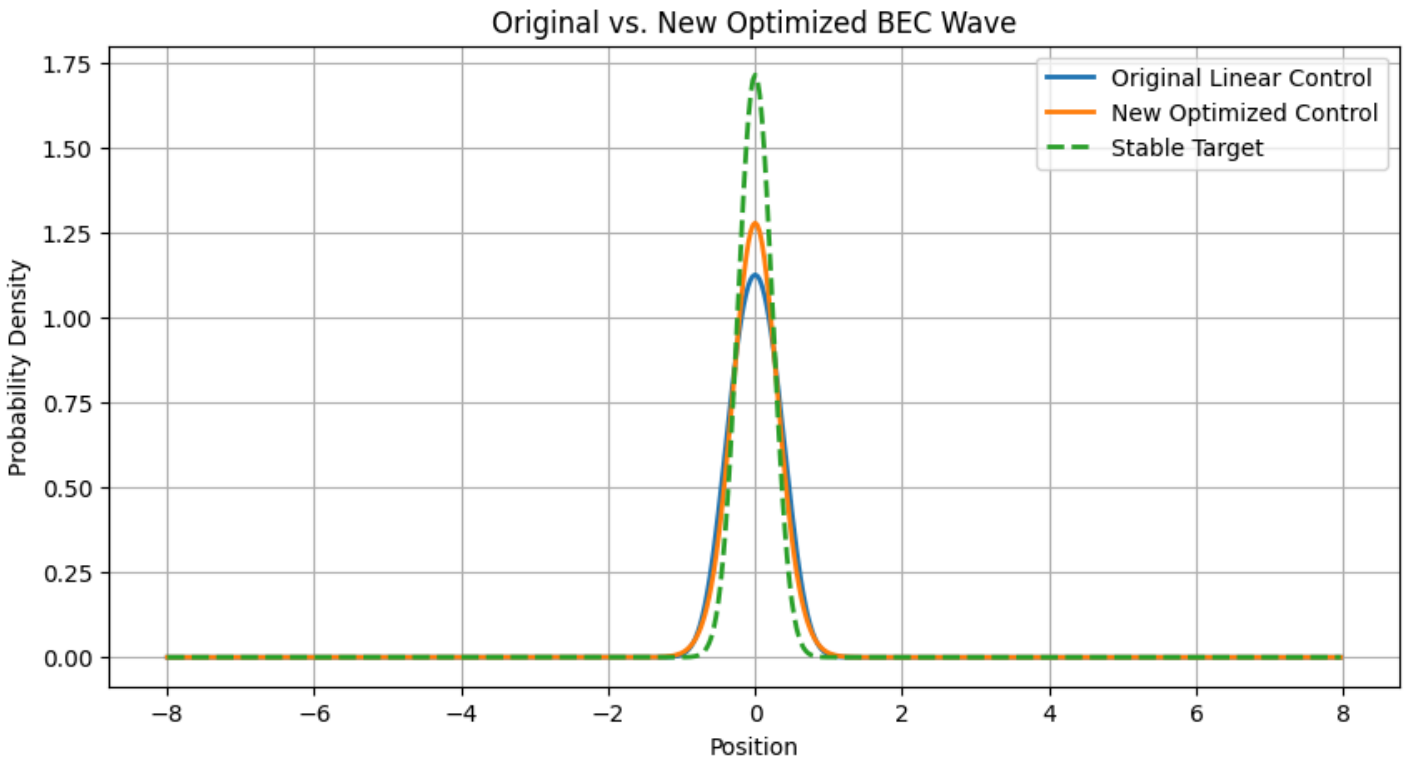
Density error measures differences in the spatial distribution, while infidelity measures differences between the quantum states, including relative phase.

For normalized states, zero infidelity means a perfect match up to an overall phase. The code also plots the original result, optimized result, and target for comparison.

Why we use it: These comparisons test whether our updated control settings produce a final state closer to the researcher’s target. Comparing the errors before and after optimization lets us measure the improvement.

```
Optimization completed!
Coefficients: [-5.31053093 -2.73345766 -1.47694838]
Final objective: 0.49303166795515646
Success: True
Optimization time: 14.98 seconds
```

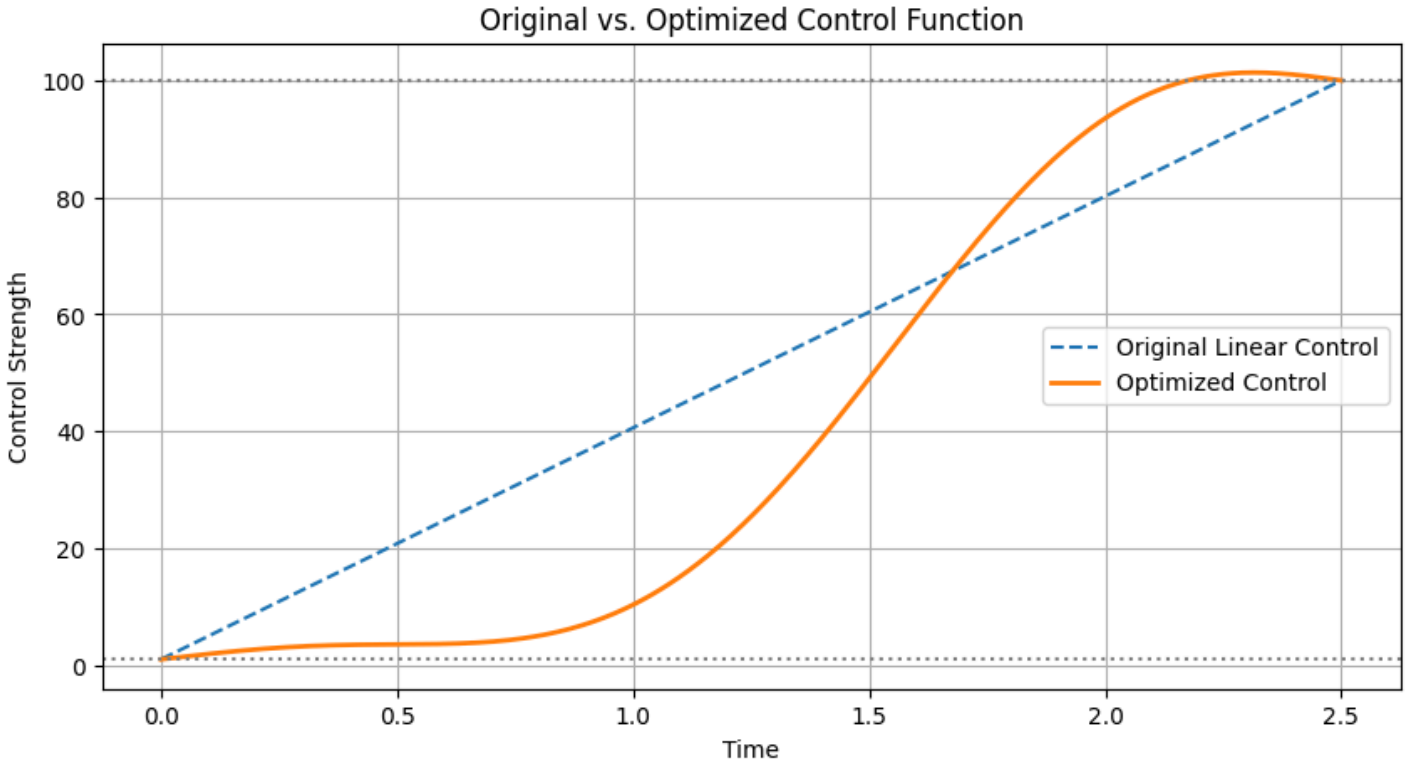
```
Original density error: 0.356551
New optimized density error: 0.262822
New infidelity: 0.08565
```



This code plots the original linear control and the newly optimized control over time. It also prints the starting, ending, minimum, and maximum control strengths.

Why we're using it: We want to verify that the optimized control starts at 1, ends at 100, and behaves reasonably throughout the simulation. If it goes outside the allowed range, we'll need to add constraints. How to explain it in your presentation After optimizing the BEC wave, we examined the control function to understand how the algorithm achieved its improved result. We compared the original linear control against the optimized control and checked its minimum and maximum values. This helped us determine whether the optimized solution respected our control requirements.

```
Starting control: 1.0
Ending control: 100.0
Minimum control: 1.0
Maximum control: 101.3558
```

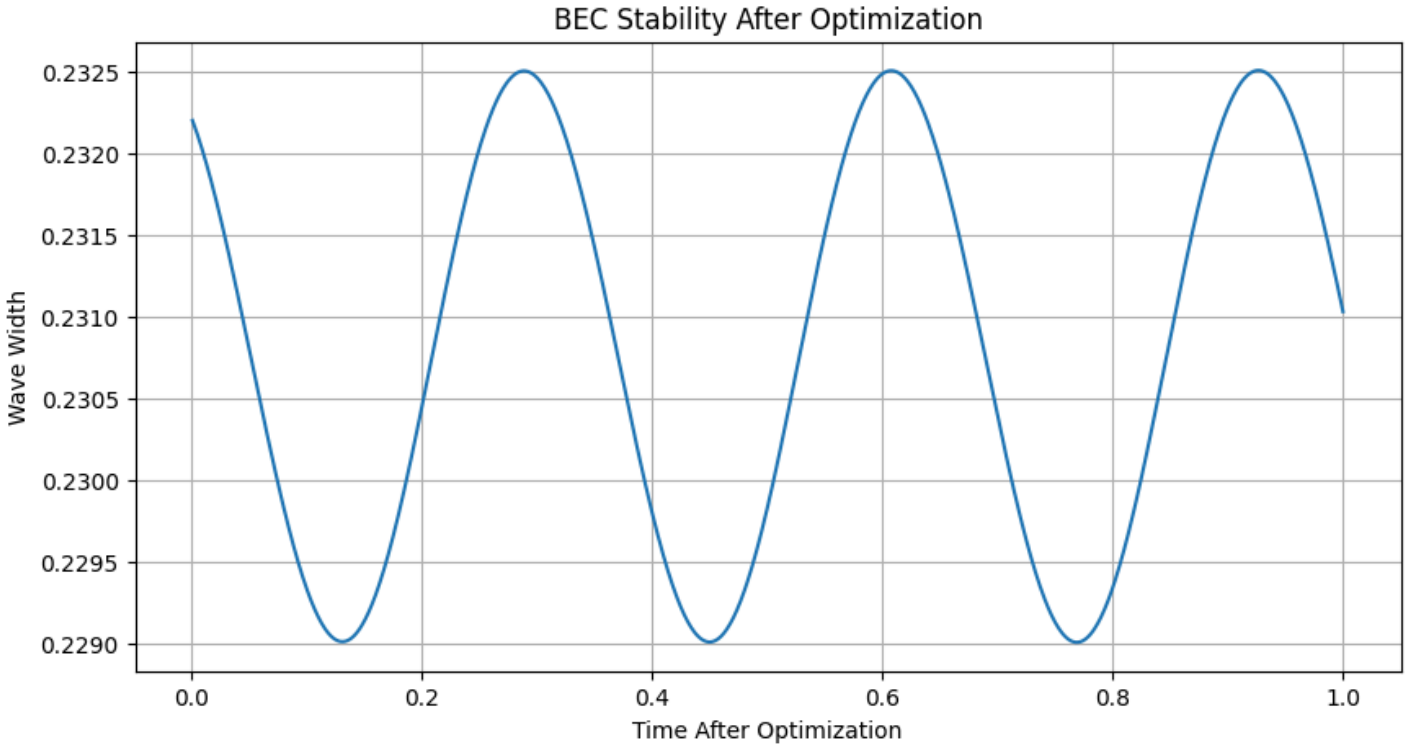


This code continues simulating the optimized BEC wave after the control reaches its final value of 100. It measures the wave's width over time and creates a graph showing whether that width stays constant or oscillates.

Our researcher wants the wave to reach a stable final state without unwanted oscillations. A small width variation would support the idea that our optimized wave is relatively stable. A large variation would indicate that the wave is still oscillating and that the control needs improvement.

Width is only one stability measurement, so we'll eventually check the full probability density as well.

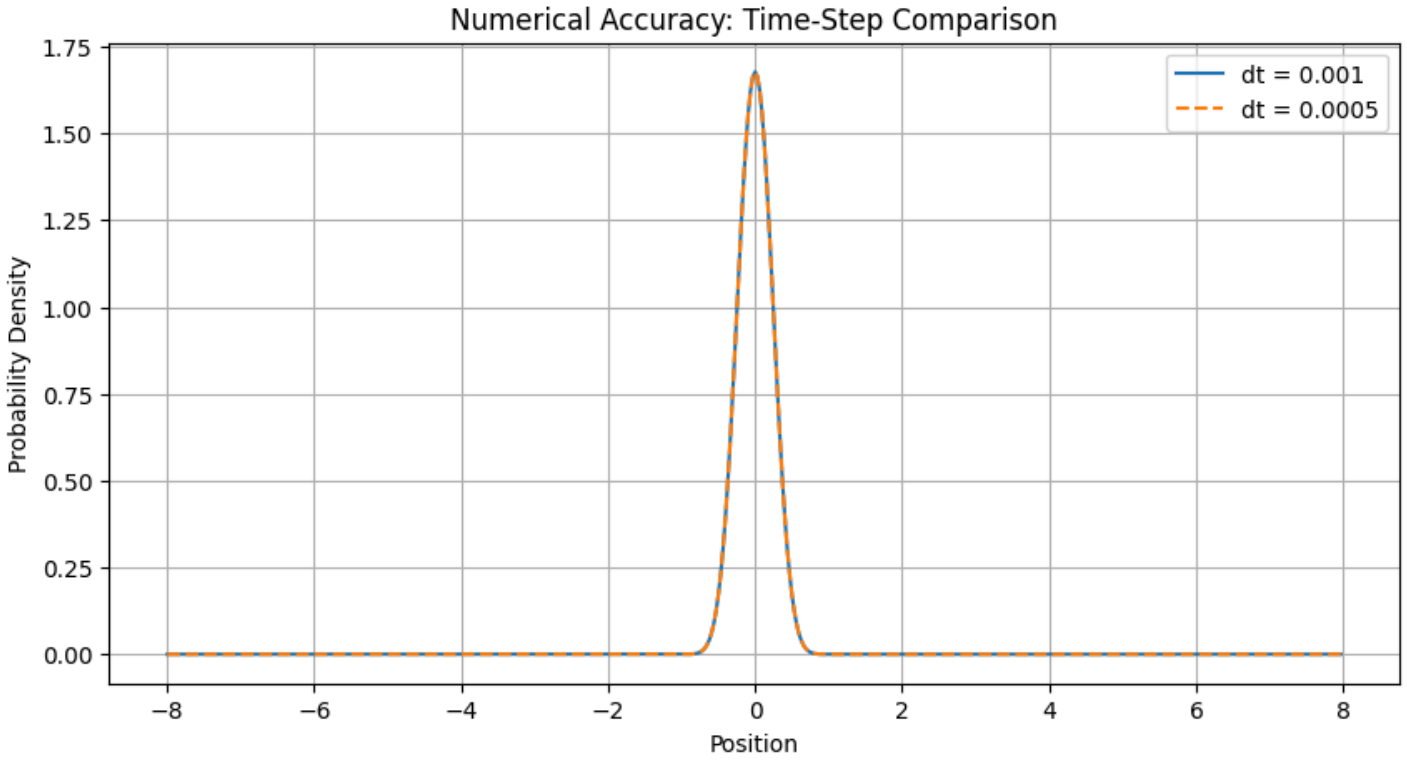
After optimizing the BEC wave, we tested whether it remained stable when the control was held constant. We continued the simulation for an additional time period and measured the wave's width at each step. This allowed us to identify whether the optimized wave continued oscillating or maintained a consistent shape. Stability is important because our goal is not only to squeeze the wave but also to reach a desired final state without unwanted movement.




```
Minimum width: 0.229008
Maximum width: 0.232504
Width variation: 0.003496
```

This code checks how sensitive the BEC simulation is to the time step by running it with the same optimized controls using steps of 0.001 and 0.0005. It calculates the difference between the final probability densities and plots them for comparison. If the difference is small and the curves overlap, that suggests the density result is consistent under time-step refinement, helping validate the traditional solver before using it to train the AI neural operator.

```
Time-step difference: 2.5976259204165597e-06
```




```
 The numerical accuracy test looks very good! The two lines overlap almost perfectly,
 which means our simulation produces nearly identical results when we use a smaller time step.
 this is a very small difference. It suggests that our numerical results are consistent when we reduce the time step. Your graph confirms this visually:
the blue and orange lines are almost indistinguishable.
 Why this matters: Before training our AI neural operator, we need confidence that the traditional solver is producing reliable results.

 Otherwise, the AI could learn from inaccurate simulation data. We still need to test spatial resolution and full wave-function accuracy, but this is a
useful first validation.
```

This code creates a dataset for training your AI model. It randomly selects 30 sets of three control coefficients, each between −20 and 20, and runs your traditional BEC solver for each set. It saves the coefficients as inputs and the resulting wave's real and imaginary components as outputs. It also measures how long data generation takes and saves everything in bec_training_data.npz. The fixed random seed makes the coefficient choices reproducible.

It generates training examples; it does not train the AI yet.

```
Before training an AI model, we need examples it can learn from.
Our traditional solver will generate these examples by running simulations with different control functions.
Each example will contain a set of control coefficients and the resulting wave function.


Training data generated!
Control data shape: (30, 3)
Wave data shape: (30, 512, 2)
Generation time: 12.51 seconds
Dataset saved as bec_training_data.npz
```

This is the next step after generating your dataset. It loads the 30 saved simulations, shuffles them reproducibly while keeping each control set paired with its wave, and divides them into:

- 20 training examples: Used to teach the model.

- 5 validation examples: Used to guide model choices during development.
- 5 testing examples: Kept aside to evaluate the finished model on unseen examples. It then converts the data into PyTorch tensors, the format your AI training code needs. The inputs are the three control coefficients; the outputs are the corresponding wave's real and imaginary components. This prepares the data—it still doesn't train the model.

```
Before building our neural operator, we need to separate the data so we can measure whether the AI can predict waves it hasn't seen before.
We'll use 20 simulations for training, 5 for validation, and 5 for testing.


Training examples: 20
Validation examples: 5
Testing examples: 5
Input shape: torch.Size([20, 3])
Output shape: torch.Size([20, 512, 2])
```

```
Generated 20/80 additional examples
Generated 40/80 additional examples
Generated 60/80 additional examples
Generated 80/80 additional examples

Training examples: 100
Validation examples: 5
Additional generation time: 31.81 seconds
Saved: bec_expanded_data.npz
The previously inspected test cases are exploratory.
```

Result explination:

Dataset	Examples	Purpose
Training	20	Teach the AI how controls affect the wave
Validation	5	Check how well the AI is learning
Testing	5	Evaluate predictions on unseen examples

Your input shape is `(20, 3)`, meaning each training example has **3 control coefficients**.

Your output shape is `(20, 512, 2)`, meaning each example contains a wave measured at **512 positions**, with real and imaginary values.

This code builds the AI model that will learn to predict the final BEC wave from three control coefficients. It combines those coefficients with 512 spatial positions, then passes them through two layers that learn broad patterns using Fourier frequencies and local features using convolution layers. At each position, it outputs two numbers: the predicted real and imaginary parts of the wave function. Finally, it creates the model and prints its number of trainable parameters. It does not train the model yet.

- This is an FNO-style model. It becomes physics-informed only if you add physics constraints during training.

```
A Fourier Neural Operator is a type of AI model that learns relationships between functions.
 In our project, we want it to learn how different control functions affect the final BEC wave.
We're going to build a small FNO-style model using PyTorch. For now, we're only creating the model, not training it yet.
```

```
Neural operator created successfully!
Trainable parameters: 35106
```

NOW To train the data to predict how the BEC wave responds to different controls

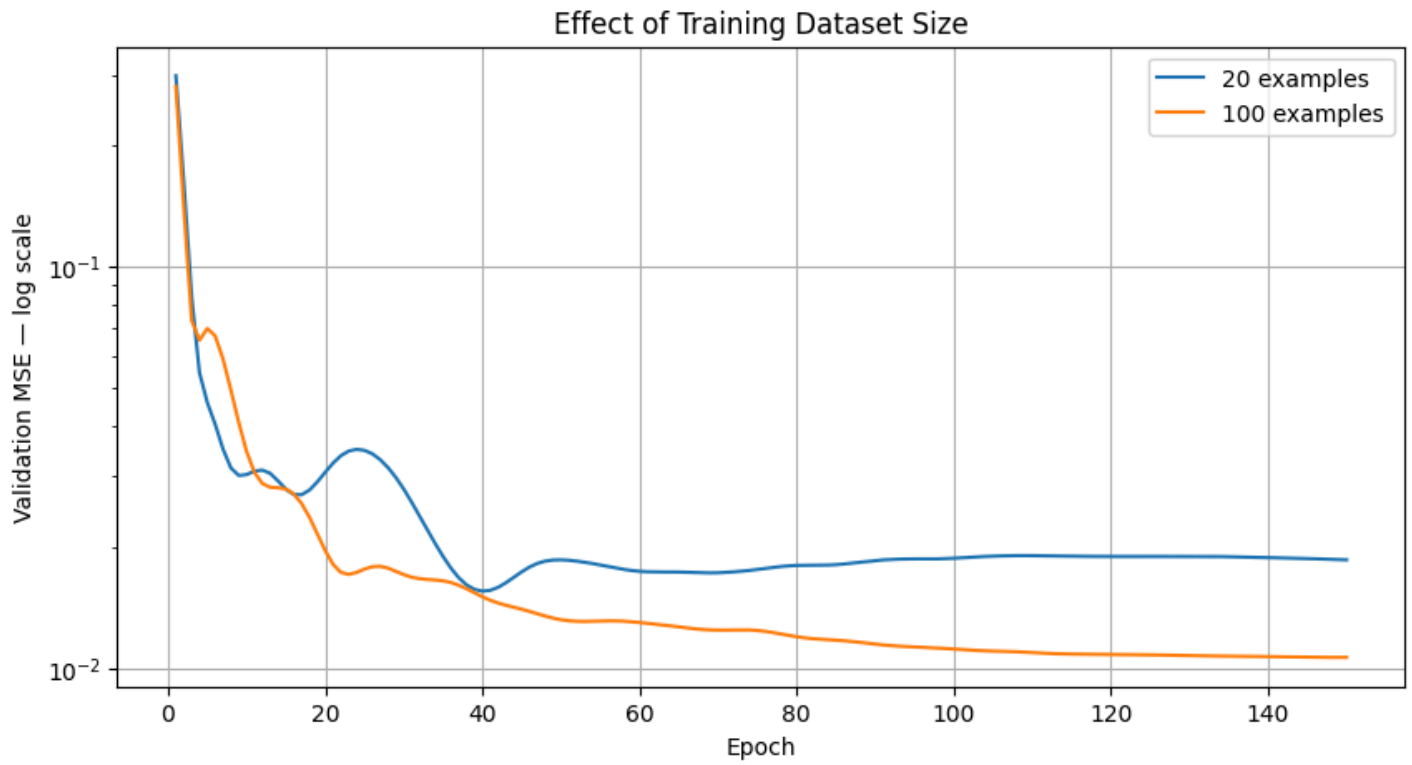
This code trains our AI model using the BEC simulations we generated earlier. The model predicts the final wave from the control settings, compares its prediction with the traditional solver's result, and adjusts itself to reduce mistakes. It repeats this process 150 times. We also check its predictions on separate validation examples to see whether it can handle examples it did not train on. Finally, the code reports the training time and plots the training and validation errors to show its progress.

Our traditional numerical solver calculates how a BEC wave evolves by solving the Gross–Pitaevskii equation. We want to determine whether an AI neural operator can learn that relationship and predict the final wave without repeatedly performing the full numerical simulation.

The training and validation losses help us measure whether the AI is learning. However, we still need to evaluate its predictions on the five test examples before deciding how accurate it is. How to explain it in your presentation We trained our Fourier Neural Operator using simulation data generated by our traditional GPE solver. The model learned to connect different control settings with their resulting BEC wave functions. We trained the model for 150 epochs and measured both training and validation errors to track its performance. Our goal was to determine whether the neural operator could accurately approximate the traditional solver while potentially reducing computation time.

```
Training with 20 examples...
Epoch 50: train MSE=0.005683, validation MSE=0.018627
Epoch 100: train MSE=0.002651, validation MSE=0.018797
Epoch 150: train MSE=0.001614, validation MSE=0.018626
Validation wave error: 74.41%
Training time: 5.63 seconds

Training with 100 examples...
Epoch 50: train MSE=0.009031, validation MSE=0.013217
Epoch 100: train MSE=0.005942, validation MSE=0.011172
Epoch 150: train MSE=0.004625, validation MSE=0.010653
Validation wave error: 55.16%
Training time: 22.91 seconds
```




```
FINAL VALIDATION COMPARISON
20 examples: MSE=0.018626, relative wave error=74.41%
100 examples: MSE=0.010653, relative wave error=55.16%
```

Effect of training dataset size We compared 20 and 100 training examples using identical initial model weights, the same validation cases, and 150 training epochs. Increasing the training set reduced validation MSE from 0.018626 to 0.010653 and mean relative wave-function error from 74.41% to 55.16%.

This provides evidence that limited training data contributes to prediction error. However, substantial error remains. The conclusion is preliminary because the experiment uses five validation cases and one model initialization. Generating the additional 80 examples took 31.81 seconds, which must be included in the surrogate's computational cost.

Now we need to measure how accurately the neural operator predicts those wave functions.

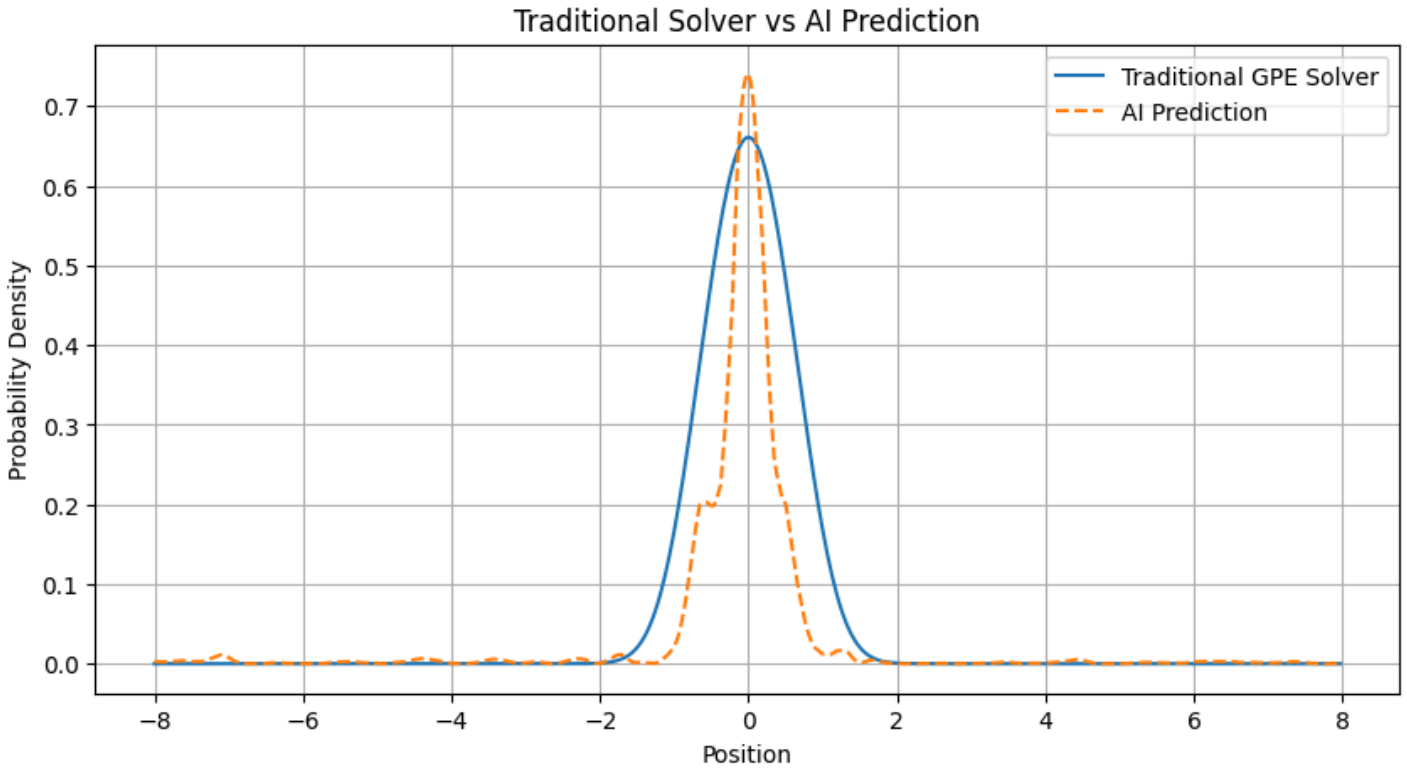
What this code does:

It gives the trained AI five control settings it did not see during training. It compares the predicted wave functions with the results from our traditional GPE solver and calculates the relative L2 error. It also plots the predicted and actual probability densities for one test example.

This is how we determine whether the neural operator can predict new BEC wave functions accurately, rather than simply memorizing training examples.

```
Relative L2 errors for each test example:
Test 1: 0.7578 (75.78%)
Test 2: 0.7799 (77.99%)
Test 3: 0.7838 (78.38%)
Test 4: 0.5620 (56.20%)
Test 5: 0.4041 (40.41%)
```

```
Average test error: 65.75%
```



After training our neural operator, we tested it using five previously unseen examples. We compared the AI-generated wave functions against our traditional GPE simulation using relative L2 error. This allowed us to evaluate how accurately the AI generalized to new control settings. We also compared probability density plots to visually identify differences between the two methods.

This code compares how long the traditional BEC solver and the trained AI model take to produce a result for the same test control settings. It runs each method several times and averages the times, then calculates how much faster the AI prediction is. It also includes the time spent generating training data and training the model to estimate the break-even point—how many predictions are needed before the AI saves time overall.

```
Next step — Compare computation times


Traditional solver: 1.185887 seconds
AI prediction: 0.001119 seconds
Prediction speedup: 1060.04x
Estimated break-even queries: 57
```

```
Test 1
Actual probability:          1.0000
Predicted probability:       0.5757
Phase-aligned wave error:    72.08%
Relative density error:      46.36%

Test 2
Actual probability:          1.0000
Predicted probability:       0.5176
Phase-aligned wave error:    77.26%
```

```
Relative density error:      57.21%

Test 3
Actual probability:          1.0000
Predicted probability:       0.2652
Phase-aligned wave error:    76.28%
Relative density error:      71.83%

Test 4
Actual probability:          1.0000
Predicted probability:       0.8267
Phase-aligned wave error:    55.62%
Relative density error:      49.83%

Test 5
Actual probability:          1.0000
Predicted probability:       0.8376
Phase-aligned wave error:    40.28%
Relative density error:      26.16%
```

```
Test 1: density error 46.36% before normalization, 54.49% after
Test 2: density error 57.21% before normalization, 87.55% after
Test 3: density error 71.83% before normalization, 57.61% after
Test 4: density error 49.83% before normalization, 42.17% after
Test 5: density error 26.16% before normalization, 33.88% after

Mean density error before: 50.28%
Mean density error after: 55.14%
```

```
Space-time data saved.
Training wave shape: (20, 101, 512)
Validation wave shape: (5, 101, 512)
Dimensions: control examples, saved times, spatial positions
Generation time: 10.79 seconds
These arrays contain complex waves, not densities.
Your existing trained model has not been changed.
```

```
Finished: Linear baseline
Finished: Validation 1
Finished: Validation 2
Finished: Validation 3
Finished: Validation 4
Finished: Validation 5

P1: NUMERICAL VERIFICATION
          Case  Time refinement wave difference (%)  512 to 1024 wave difference (%)  1024 to 2048 wave difference (%)  Density difference vs refined
solve (%)  512-grid evolution time (s)
Linear baseline                           0.003250                             0.0                              0.0
0.001131                    0.536788
   Validation 1                           0.002889                             0.0                              0.0
0.002110                    0.288587
   Validation 2                           0.007970                             0.0                              0.0
0.001191                    0.256518
   Validation 3                           0.001836                             0.0                              0.0
0.002080                    0.251989
   Validation 4                           0.004145                             0.0                              0.0
0.000926                    0.253667
   Validation 5                           0.004086                             0.0                              0.0
0.002591                    0.261058

Differences measure sensitivity to numerical refinement.
The refined solution is not an exact analytical solution.
Runtime measures the evolution loop, excluding setup.
Existing models and simulation settings are unchanged.
```

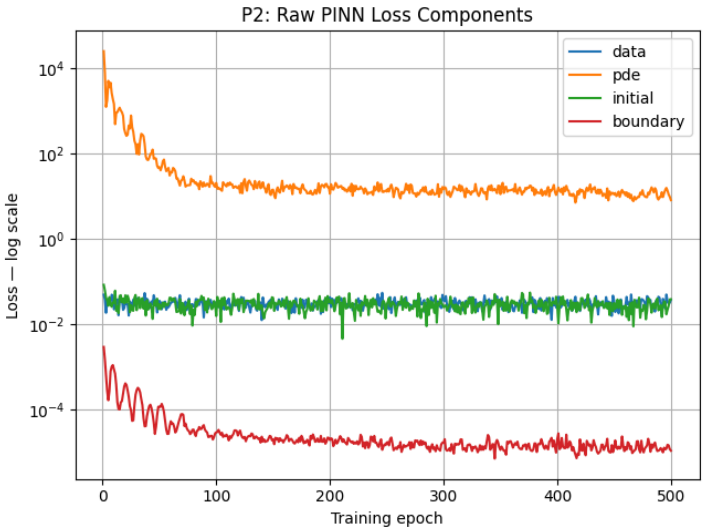
```
Space-time model created.
Trainable parameters: 8834
Inputs: x, t, and three control coefficients.
```

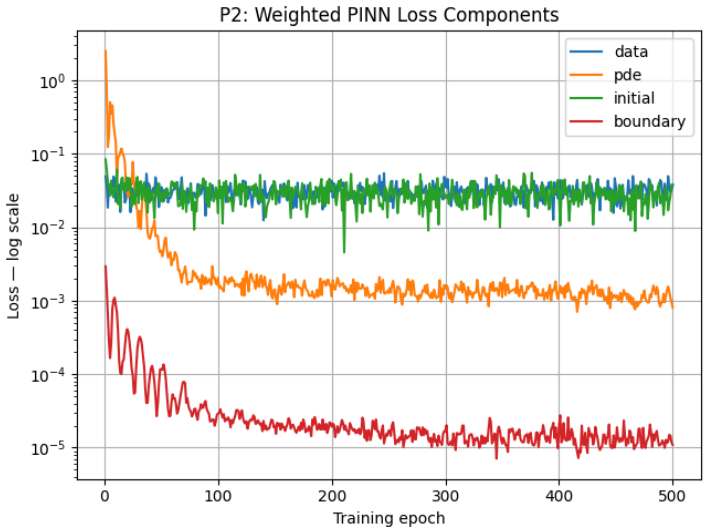
```
Outputs: real and imaginary wave components.
Training has not started.
The existing FNO is unchanged.
```

```
data loss: 3.683461e-02
pde loss: 1.910550e+04
initial loss: 5.310199e-02
boundary loss: 2.758105e-03

All losses finite: True
All gradients finite: True
Model weights have not been updated.
```

```
Epoch 1/500 | data=4.914e-02 | pde=2.491e+04 | ic=8.373e-02 | bc=2.931e-03 | validation MSE=3.631e-02
Epoch 50/500 | data=3.531e-02 | pde=4.796e+01 | ic=2.942e-02 | bc=1.172e-04 | validation MSE=3.276e-02
Epoch 100/500 | data=2.984e-02 | pde=2.042e+01 | ic=1.558e-02 | bc=2.920e-05 | validation MSE=3.274e-02
Epoch 150/500 | data=3.723e-02 | pde=1.420e+01 | ic=4.135e-02 | bc=1.985e-05 | validation MSE=3.275e-02
Epoch 200/500 | data=4.278e-02 | pde=1.143e+01 | ic=3.737e-02 | bc=1.929e-05 | validation MSE=3.270e-02
Epoch 250/500 | data=2.930e-02 | pde=1.280e+01 | ic=2.847e-02 | bc=1.694e-05 | validation MSE=3.273e-02
Epoch 300/500 | data=3.786e-02 | pde=1.462e+01 | ic=2.498e-02 | bc=1.665e-05 | validation MSE=3.270e-02
Epoch 350/500 | data=2.817e-02 | pde=1.117e+01 | ic=2.656e-02 | bc=1.135e-05 | validation MSE=3.267e-02
Epoch 400/500 | data=2.640e-02 | pde=1.261e+01 | ic=2.683e-02 | bc=2.016e-05 | validation MSE=3.267e-02
Epoch 450/500 | data=3.600e-02 | pde=1.094e+01 | ic=3.132e-02 | bc=1.244e-05 | validation MSE=3.264e-02
Epoch 500/500 | data=3.712e-02 | pde=8.085e+00 | ic=3.836e-02 | bc=1.090e-05 | validation MSE=3.262e-02
```

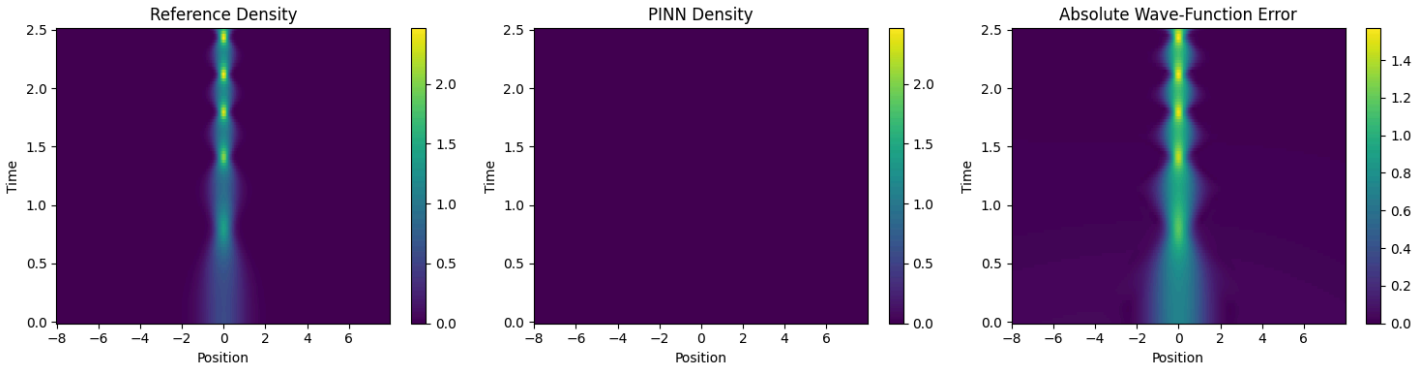


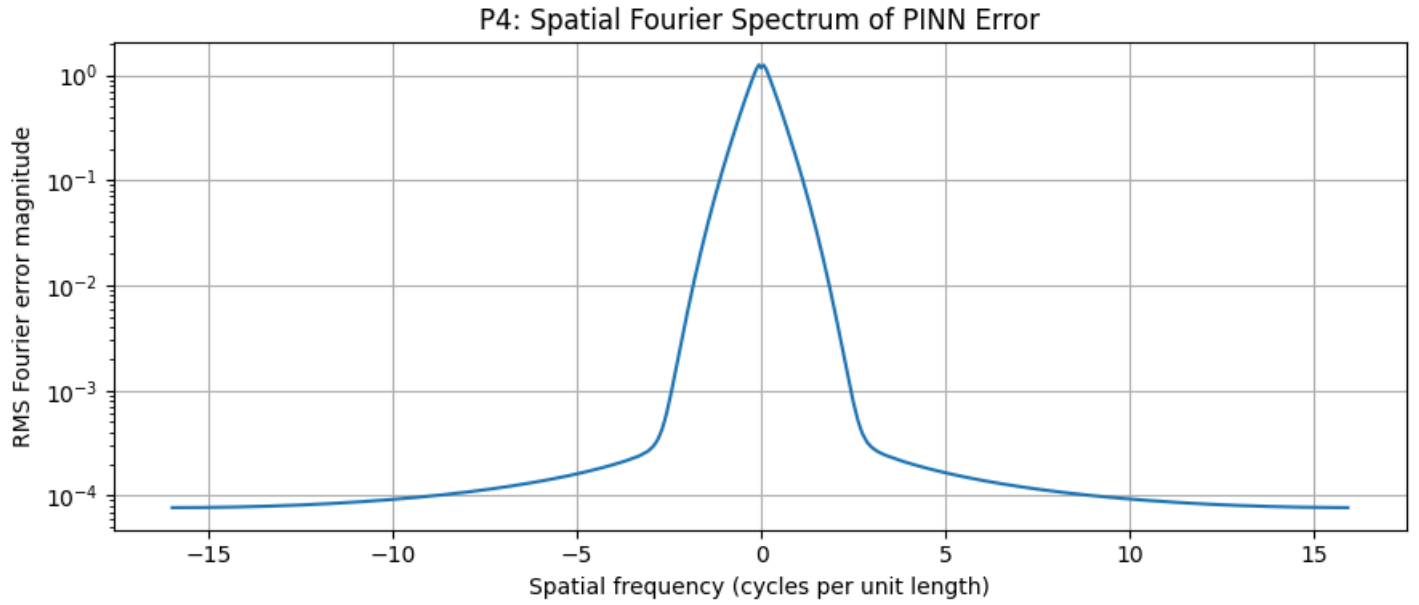



```
PINN training completed.
Selected checkpoint epoch: 500
Best sampled validation MSE: 0.03262462094426155
Training time: 10.92 seconds
Existing FNO remains unchanged.
```

```
Validation 1: space-time wave error=98.99% | predicted probability range=0.0001 to 0.0227
Validation 2: space-time wave error=99.80% | predicted probability range=0.0001 to 0.0050
Validation 3: space-time wave error=99.63% | predicted probability range=0.0001 to 0.0056
Validation 4: space-time wave error=99.50% | predicted probability range=0.0000 to 0.0147
Validation 5: space-time wave error=99.79% | predicted probability range=0.0000 to 0.0042

Mean space-time validation error: 99.54%
```





Physics-informed model failure

The PINN's PDE and boundary losses decreased during training, but its mean relative space–time validation error remained 99.54%. Predicted total probabilities stayed below 0.023 instead of the reference value of 1. These results indicate near-zero predictions. The zero function satisfies the homogeneous GPE and periodic boundary conditions, illustrating why small physics losses alone do not guarantee an accurate solution. This experiment does not yet establish why training favored this solution.

```
Epoch 1/500 | data MSE=4.914e-02 | validation MSE=3.639e-02
Epoch 50/500 | data MSE=2.977e-02 | validation MSE=3.250e-02
Epoch 100/500 | data MSE=2.837e-02 | validation MSE=3.236e-02
Epoch 150/500 | data MSE=3.088e-02 | validation MSE=3.262e-02
Epoch 200/500 | data MSE=3.957e-02 | validation MSE=3.232e-02
Epoch 250/500 | data MSE=2.408e-02 | validation MSE=3.243e-02
Epoch 300/500 | data MSE=3.635e-02 | validation MSE=3.247e-02
Epoch 350/500 | data MSE=2.112e-02 | validation MSE=3.211e-02
Epoch 400/500 | data MSE=2.552e-02 | validation MSE=3.194e-02
Epoch 450/500 | data MSE=2.369e-02 | validation MSE=3.169e-02
Epoch 500/500 | data MSE=3.037e-02 | validation MSE=3.124e-02

Validation 1: wave error=94.58% | probability range=0.0040 to 0.2193

Validation 2: wave error=100.03% | probability range=0.0055 to 0.1373

Validation 3: wave error=97.16% | probability range=0.0045 to 0.2522

Validation 4: wave error=96.66% | probability range=0.0056 to 0.1934
```

```
Validation 5: wave error=98.53% | probability range=0.0054 to 0.1704

CONTROLLED COMPARISON
Physics-informed mean space-time error: 99.54%
Data-only mean space-time error: 97.39%
Data-only selected epoch: 500
Data-only training time: 1.02 seconds
```

Controlled comparison: physics-informed versus data-only training We trained the same space–time architecture from identical initial weights using either physics-informed losses or data loss alone. Both experiments used the same training controls, learning rate, number of updates, and validation checkpoint rule.

Mean relative space–time validation error was 99.54% for physics-informed training and 97.39% for data-only training. Both models substantially underestimated total probability. Removing the physics losses did not restore accurate predictions. Therefore, this experiment does not isolate physics-loss imbalance as the sole cause. Further checks are needed to distinguish training limitations, representation limitations, and sampling effects.

```
Evaluating all 20 training trajectories...

PHYSICS-INFORMED MODEL
Training wave error: 99.45%
Validation wave error: 99.54%
Mean training probability: 0.0023

DATA-ONLY MODEL
Training wave error: 96.64%
Validation wave error: 97.39%
Mean training probability: 0.0602

Reference total probability is approximately 1.
No model weights were changed.
```

```
TIME HORIZON: 0.1
  Data only: epoch 250/500
  Data only: epoch 500/500
  Validation wave error: 9.09%
  Physics-informed: epoch 250/500
  Physics-informed: epoch 500/500
  Validation wave error: 10.81%

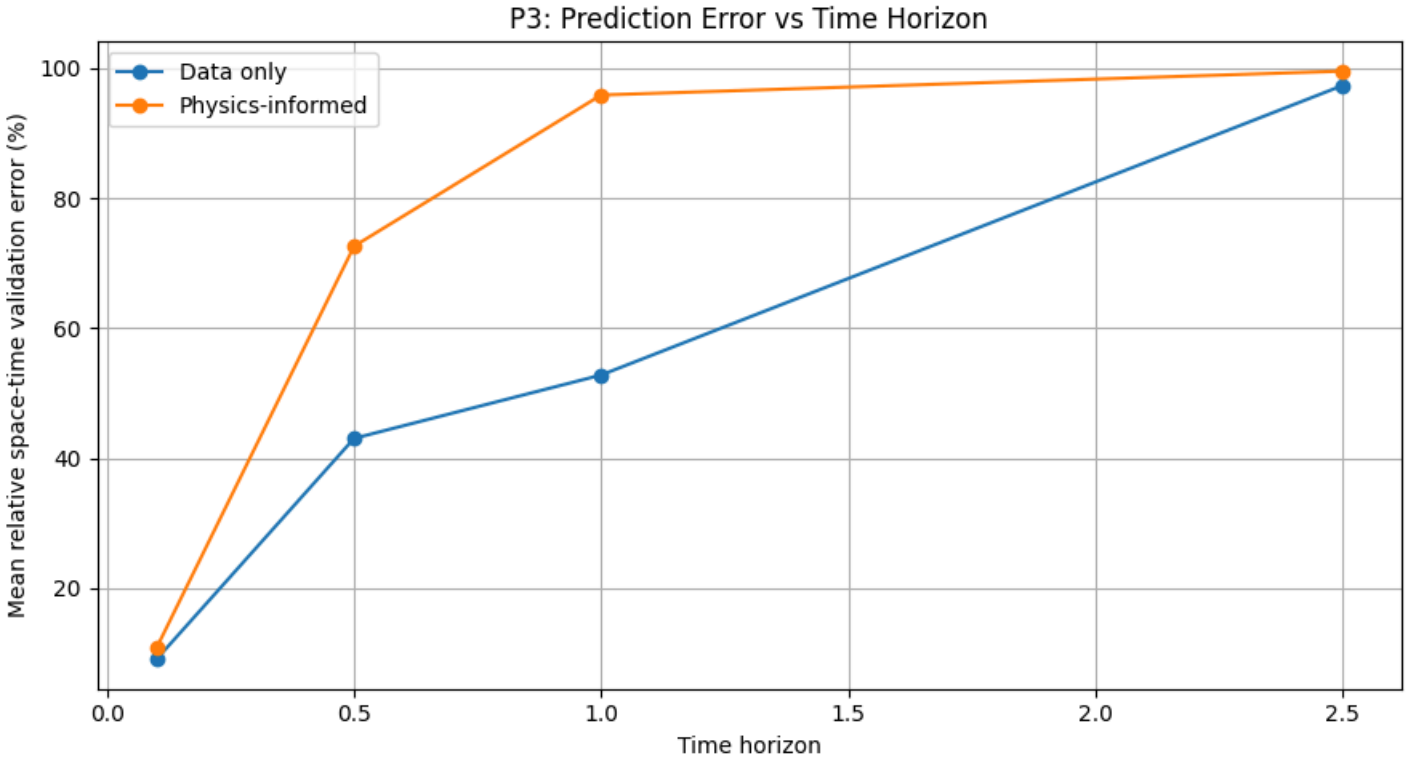
TIME HORIZON: 0.5
  Data only: epoch 250/500
  Data only: epoch 500/500
  Validation wave error: 43.06%
  Physics-informed: epoch 250/500
  Physics-informed: epoch 500/500
  Validation wave error: 72.61%

TIME HORIZON: 1.0
  Data only: epoch 250/500
  Data only: epoch 500/500
  Validation wave error: 52.80%
  Physics-informed: epoch 250/500
  Physics-informed: epoch 500/500
  Validation wave error: 95.85%

TIME HORIZON: 2.5
  Data only: epoch 250/500
  Data only: epoch 500/500
  Validation wave error: 97.30%
  Physics-informed: epoch 250/500
  Physics-informed: epoch 500/500
  Validation wave error: 99.52%

P3: TIME-HORIZON RESULTS
 Horizon         Approach  Validation wave error (%)  Mean predicted probability  Training time (s)  Five-trajectory evaluation time (s)
     0.1        Data only                     9.0913                      0.9720             2.9854                               0.0360
     0.1 Physics-informed                    10.8135                      0.9508             9.2245                               0.0242
```

```
0.5        Data only          43.0598          0.7121          1.4434          0.2385
0.5 Physics-informed          72.6076          0.3016         12.0059          0.2000
1.0        Data only          52.8016          0.6419          1.5193          0.3566
1.0 Physics-informed          95.8485          0.0417          9.1529          0.1597
2.5        Data only          97.2959          0.0459          1.0697          0.4257
2.5 Physics-informed          99.5200          0.0023          9.4951          0.3753
```




```
Existing FNO and PINN models remain unchanged.
Results use final checkpoints after 500 updates.
Errors are evaluated on the same five validation controls.
```

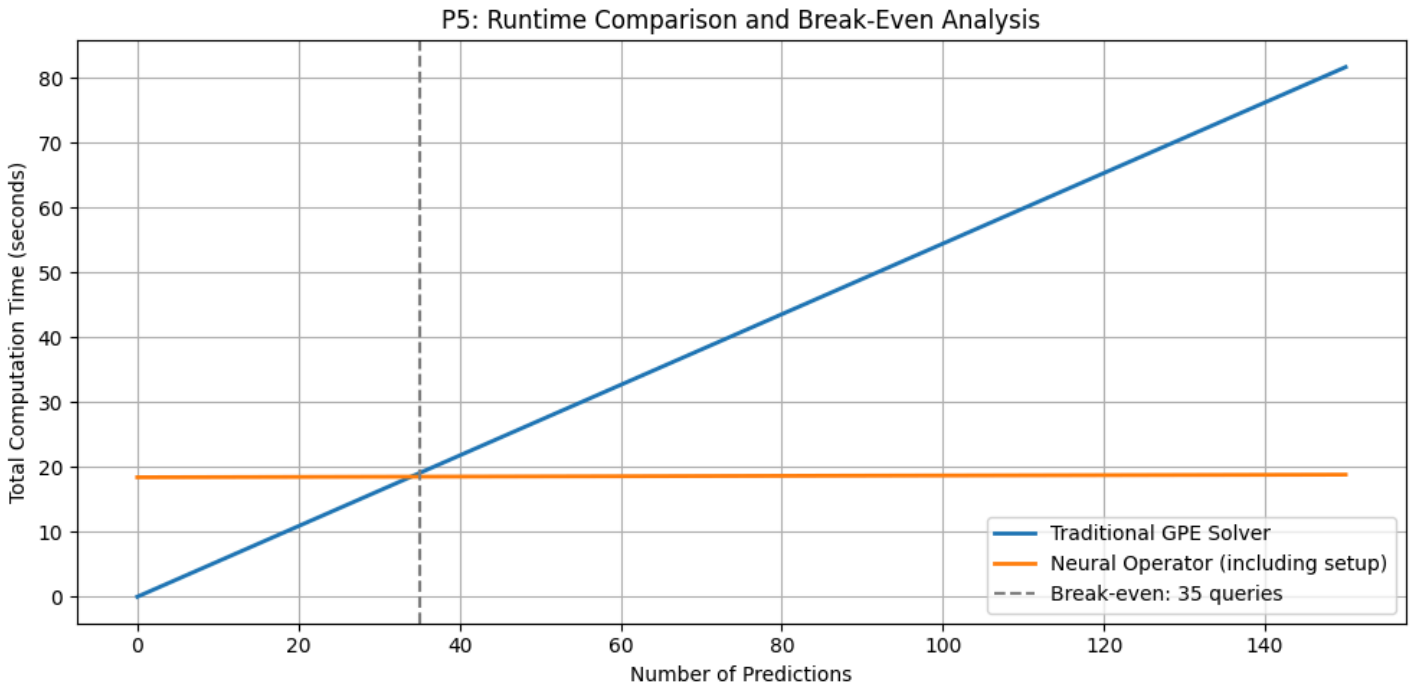
P3: Effect of time horizon We used time horizon as the difficulty knob, testing 0.1, 0.5, 1.0, and 2.5 dimensionless time units. Each experiment used the same architecture, initial weights, training controls, learning rate, and 500 updates. Both approaches received identical sampled data points. The control schedule was unchanged; longer horizons included more of the same evolution.

Data-only validation error increased from 9.09% to 97.30%, while physics-informed error increased from 10.81% to 99.52%. The physics-informed model's mean predicted probability decreased from 0.9508 to 0.0023. These results identify increasing trajectory duration as a contributor to failure under the tested training budget. The physics-informed formulation performed worse at intermediate horizons and approached the zero solution at long horizons. However, this experiment does not isolate temporal oscillations, sampling density, and optimization difficulty from one another. It does not prove a causality failure or that longer training would be ineffective. The conclusion is limited to one initialization, five validation controls, and the chosen loss weights.

This code creates a graph comparing the total time needed to make up to 150 predictions using the traditional BEC solver and the AI model. The AI's total includes generating its training data and training it, plus prediction time. The dashed line marks the break-even point, where the AI begins saving time overall. Finally, it displays the graph and saves

it as P5_runtime_comparison.png for your report. These lines are estimates based on your previously measured average runtimes; the code does not run new simulations.

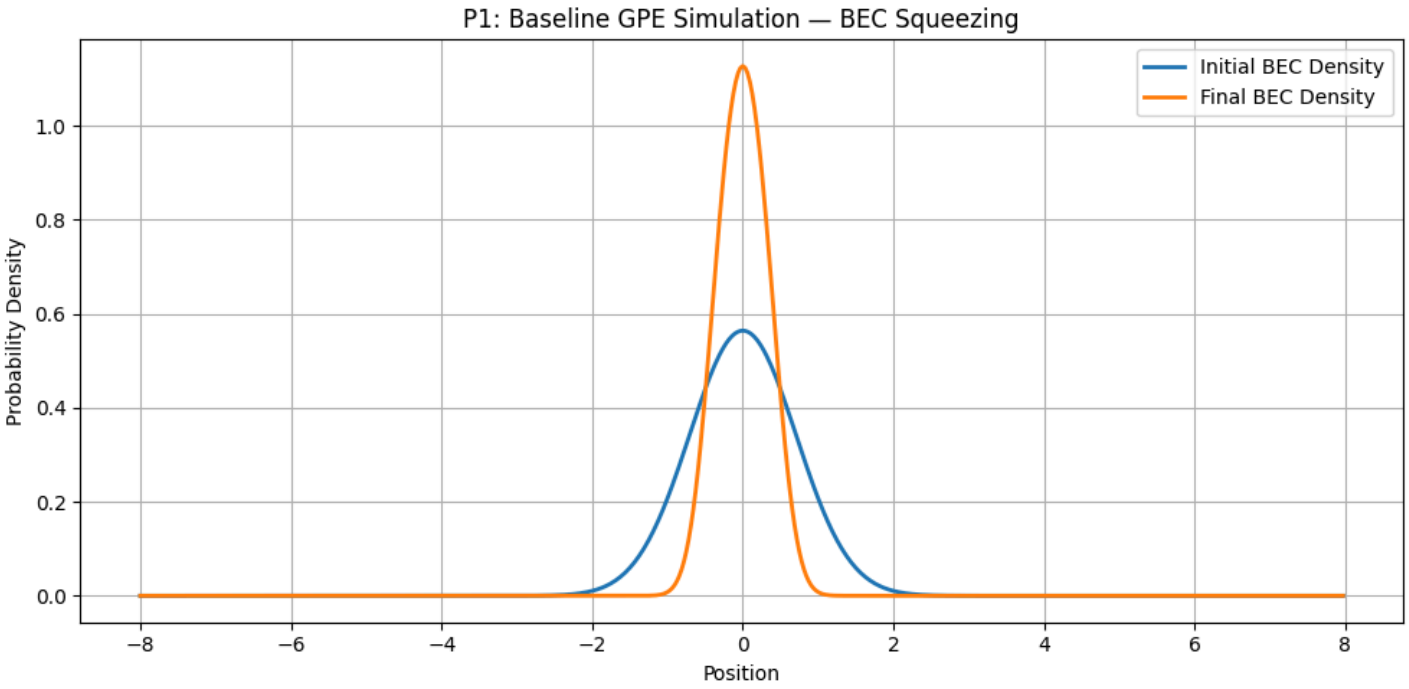
```
Create your required runtime graph
```




```
P5 graph saved successfully!
```

This code plots the initial and final BEC probability densities from the traditional GPE solver, showing how the density changes during the simulation. It displays the graph and saves it as P1_baseline_simulation.png for the report.

```
Save P1: Baseline GPE simulation
```




```
P1 graph saved successfully!
```

P1 graph shows how the Bose–Einstein condensate changes when we increase the trapping strength.

Blue line: The initial condensate has a wider probability distribution.

Orange line: The final condensate is narrower and more concentrated around the center.

Result: The condensate's width decreased by approximately 52.56% in our baseline simulation. This demonstrates that our traditional GPE solver successfully simulated the squeezing behavior. How to explain it in your presentation Our first experiment used a traditional numerical solver to simulate Bose–Einstein condensate squeezing. As the trapping strength increased, the condensate became narrower and more concentrated near the center. We measured a width reduction of approximately 52.56%. This simulation established the baseline for our later optimization and AI experiments.

This code uses the trained AI model with SciPy's optimizer to search for control settings predicted to produce a wave close to the target. It then checks those settings with the traditional GPE solver and compares the resulting density error with your previous traditional optimization result. This tests whether the AI-selected controls work when applied to the actual simulation.

```
Running traditional optimization...
Running AI-assisted optimization...

============================================================
MATCHED TRADITIONAL VS AI COMPARISON
============================================================

Traditional control coefficients:
[-19.99999533  -3.95512917   0.985838  ]

AI-selected control coefficients:
[ 7.2113497  -9.69234177  3.50331095]
```

```
Solver-verified density L2 error:
Traditional: 0.253400
AI-assisted: 0.254834

Solver-verified objective, including coefficient penalty:
Traditional: 0.052612
AI-assisted: 0.122382

AI-predicted objective:
0.081621

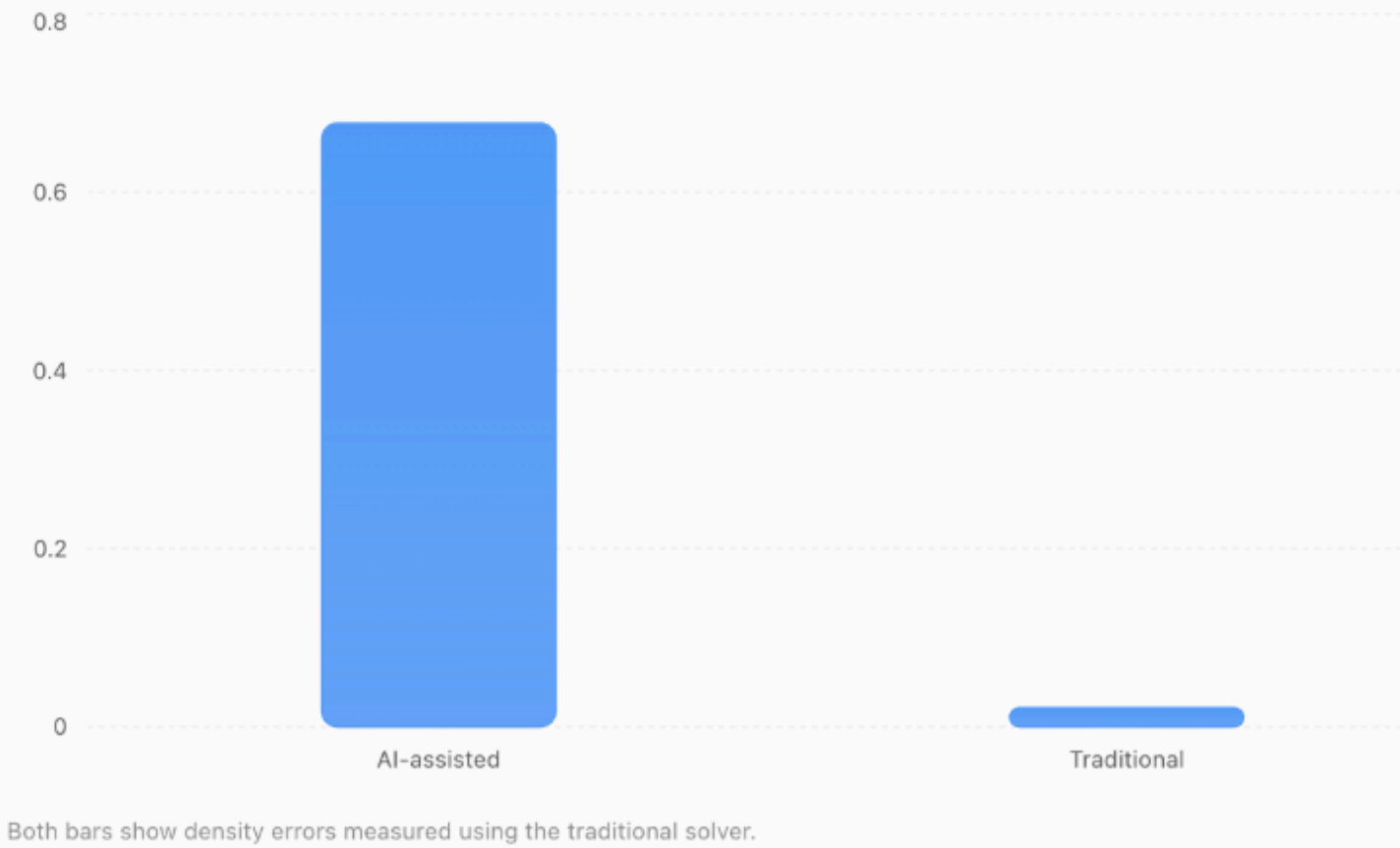
Optimization time, excluding final verification:
Traditional: 121.88 seconds
AI-assisted: 0.21 seconds

Objective evaluations:
Traditional: 323
AI-assisted: 104

Optimizer success:
Traditional: True
AI-assisted: True

Optimizer messages:
Traditional: Optimization terminated successfully.
AI-assisted: Optimization terminated successfully.

Times exclude AI data generation and training.
Coefficient bounds match the AI training range [-20, 20].
These bounds do not guarantee bounds on the control u(t).
============================================================
```



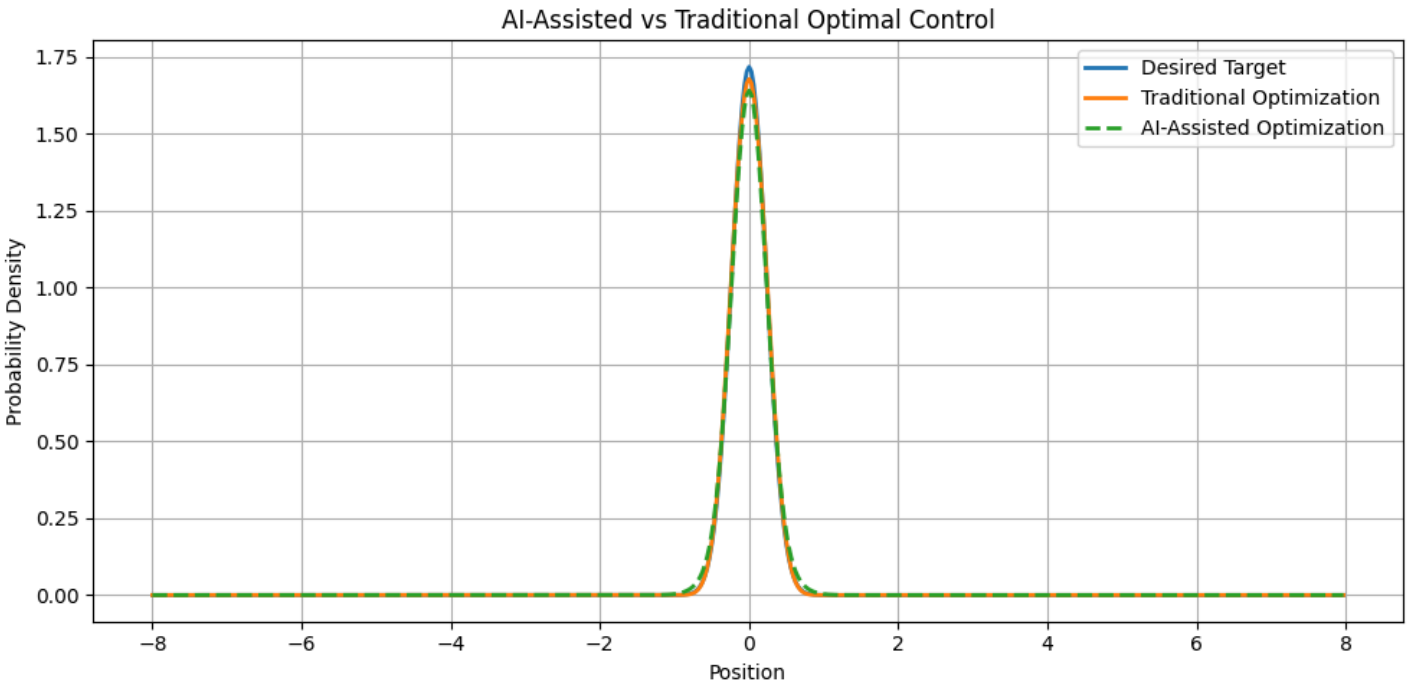
We compared traditional and AI-assisted optimization using the same objective, coefficient bounds, starting point, and optimizer settings. Both sets of selected controls were evaluated using the conventional GPE solver. The verified density errors were similar: 0.253400 for traditional optimization and 0.254834 for AI-assisted optimization. However, the verified objective—including wave-function infidelity and a coefficient penalty—was lower for traditional optimization: 0.052612

compared with 0.122382. The AI search took 0.21 seconds, compared with 121.88 seconds for traditional optimization. These times exclude neural-model training, data generation, and final verification. The AI also underestimated the objective of its selected controls, predicting 0.081621 instead of the verified 0.122382. This experiment demonstrates fast surrogate-assisted search, but further accuracy improvements and controlled experiments are needed before recommending it for reliable control optimization

---

This code plots the desired BEC target density alongside the results from traditional and AI-assisted optimization. It lets us visually compare how closely each method matches the target. It then displays the graph and saves it as optimal_control_comparison.png for the report.

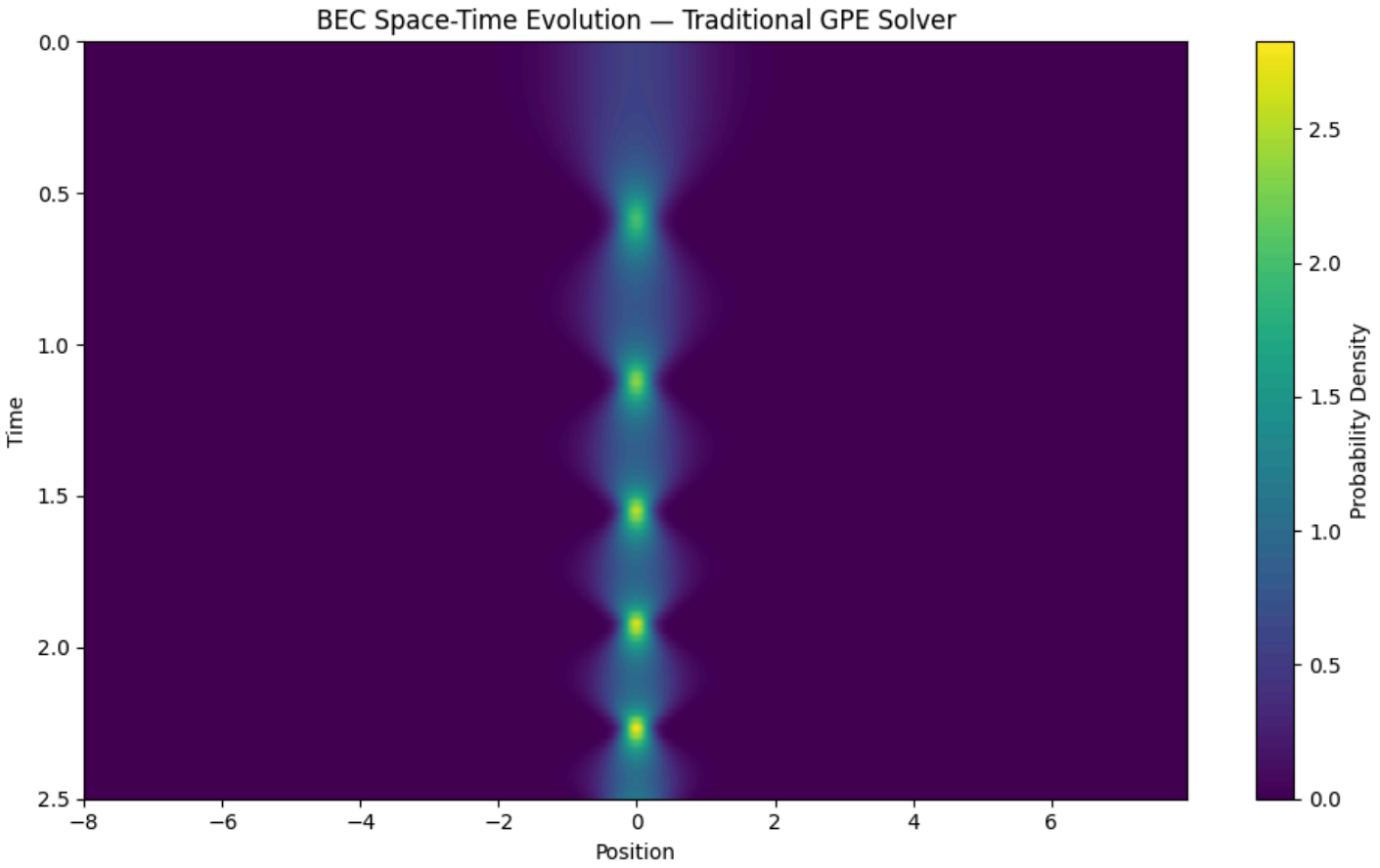
```
Save our optimization comparison graph
```




```
Optimization comparison graph saved successfully!
```

This code uses the saved simulation history to create a heatmap showing how the BEC's probability density changes across position and time. Brighter colors indicate higher density. It displays the graph and saves it as P4c_spacetime_evolution.png. This shows the traditional solver's evolution, not the AI's space-time prediction error.

```
Complete the space-time diagnostic
History shape: (101, 512)
```




```
Space-time evolution graph saved successfully!
```

The heatmap shows the Bose–Einstein condensate becoming concentrated near the center as the trapping strength increases.

- Dark purple: Areas with very low probability density.
- Green and yellow: Areas with higher probability density.
- Bright spots near the center: Periods when the condensate becomes especially concentrated. Notice that the bright spots appear repeatedly rather than forming one continuous bright line. This suggests the condensate undergoes oscillations as it is squeezed. How to explain this in your presentation We used a space-time heatmap to visualize how the Bose–Einstein condensate evolved under the traditional GPE solver. The simulation recorded 101 snapshots across 512 spatial positions. As the trapping strength increased, the condensate became more concentrated near the center, with visible oscillations during the process.

```
================================================================
FINAL RESULTS AND COMPUTATIONAL COST
================================================================

COMPUTING ENVIRONMENT
Operating system: Linux
Processor: x86_64
PyTorch version: 2.11.0+cpu
PyTorch CPU threads: 1
FNO device: cpu
PINN device: cpu

P1: CONVENTIONAL SOLVER VERIFICATION
Maximum time-refinement wave difference: 0.007970%
```

```
Maximum density difference against refined solve: 0.002591%

MAIN SPACE-TIME MODEL RESULTS
           Model  Training wave error (%)  Validation wave error (%)  Training time (s)
Physics-informed                  99.4451                    99.5413            10.9225
       Data only                  96.6369                    97.3898             1.0243

Space-time reference-data generation: 10.79 seconds

P3/P5: TIME-HORIZON EXPERIMENT
 Horizon         Approach  Validation wave error (%)  Mean predicted probability  Training time (s)  Five-trajectory evaluation time (s)
     0.1        Data only                     9.0913                      0.9720             2.9854                               0.0360
     0.1 Physics-informed                    10.8135                      0.9508             9.2245                               0.0242
     0.5        Data only                    43.0598                      0.7121             1.4434                               0.2385
     0.5 Physics-informed                    72.6076                      0.3016            12.0059                               0.2000
     1.0        Data only                    52.8016                      0.6419             1.5193                               0.3566
     1.0 Physics-informed                    95.8485                      0.0417             9.1529                               0.1597
     2.5        Data only                    97.2959                      0.0459             1.0697                               0.4257
     2.5 Physics-informed                    99.5200                      0.0023             9.4951                               0.3753

SUPPLEMENTARY FNO RESULTS
Mean exploratory final-time wave error: 65.75%
Conventional final-time solve: 0.865068 s
FNO prediction: 0.001127 s
Single-query inference speedup: 767.83x
FNO data generation plus 100-example training: 67.23 s
Timing-only break-even queries: 78

INTERPRETATION
Timing-only break-even does not establish useful accuracy.
Setup cost excludes earlier development experiments.
FNO errors describe final-time predictions.
PINN errors describe complete space-time trajectories.
Their error percentages are not directly interchangeable.
No models were retrained or modified.
```

Computational cost (P5) Experiments ran on a Linux x86-64 CPU environment using PyTorch 2.11.0 with one CPU thread. The main PINN trained in 10.92 seconds; generating its reference trajectories took 10.79 seconds. The supplementary FNO averaged 65.75% relative final-time wave error on five exploratory cases. Its measured inference speedup was approximately 768×, with a timing-only break-even estimate of 78 queries after including data generation and training. These timing benefits do not establish practical usefulness because prediction error remains substantial. Runtime measurements vary with the execution environment.

Conclusion and limitations The conventional solver showed small differences under numerical refinement. In contrast, both space–time neural models struggled to fit the full trajectories and substantially underestimated total probability. Shortening the time horizon improved accuracy. Physics-informed validation error increased from 10.81% at a horizon of 0.1 to 99.52% at 2.5. Data-only training also deteriorated, showing that physics losses alone do not explain the failure. These results identify sensitivity to trajectory duration under a fixed training budget. They do not isolate representation, optimization, sampling, or temporal oscillations as the underlying cause. Conclusions are limited by one initialization, five validation controls, and the chosen architecture and loss weights. We recommend retaining the conventional solver for reliable control evaluation. A useful next experiment would test time-window training against full-horizon training under matched computational budgets.